Taller 3 - IA

Mariana Liévano - 202220117
Angela Jiménez - 202210989
Luis Medina - 202321015

Punto 1

A).

Se declara incendio como I y alarma como A.

$$P(I = 1) = 0.10 \ , \ P(I = 0) = 0.90$$

Las probabilidades condicionales y sus complementos son:

$$P(A = 1 \mid I = 1) = 0.90 \, , P(A = 1 \mid I = 1) = 0.90$$
$$P(A = 1 \mid I = 0) = 0.05, \ P(A = 0 | I = 0) = 0.95$$

Incendio	Alarma	Cálculo	Probabilidad Conjunta
0	0	$0.90(0.95)$	0.855
0	1	$0.90(0.05)$	0.045
1	0	$0.10(0.10)$	0.010
1	1	$0.10(0.90)$	0.090

Para obtener la probabilidad de alarma, sumamos las dos filas donde $A = 1$:

$$P(A = 1) = \ 0.135$$

Ahora aplicamos Bayes:

$$P(I = 1 | \ A = 1) = \ \frac{P(A=1|I=1) \ P(I=1)}{P(A=1)} = \frac{2}{3}$$

Punto 2

A).

Verificación con sensor_basico

Se consulta la variable Incendio utilizando como evidencia Alarma=1. Los resultados obtenidos mediante enumeración son:

Resultado	Probabilidad obtenida
Incendio=0	0.333333
Incendio=1	0.666667

Este resultado coincide con el cálculo manual realizado en el punto 1 mediante la regla de Bayes

$$P(I = 1 | A = 1) = \frac{P(A=1|I=1)\,P(I=1)}{P(A=1)} = \frac{2}{3}$$

Se consulta la probabilidad de incendio bajo tres condiciones diferentes de evidencia.

Resultado	Probabilidad obtenida
Incendio=0	0.934840
Incendio=1	0.065160

Caso	Evidencia	P(Incendio=1)
Sin evidencia	Ninguna	0.065160
Alarma	Alarma=1	0.329517
Alarma y falla	Alarma=1, Falla=1	0.076831

Resultados obtenidos:
Bosque sin evidencia

Resultado	Probabilidad obtenida
Incendio=0	0.934840
Incendio=1	0.065160

Bosque con alarma

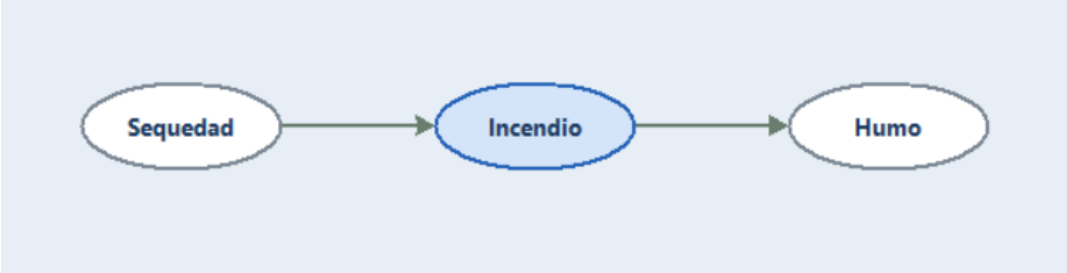
Resultado	Probabilidad obtenida
Incendio=0	0.670483
Incendio=1	0.329517

Bosque con alarma y falla

Resultado	Probabilidad obtenida
Incendio=0	0.923169
Incendio=1	0.076831

Al analizar los resultados se observa que, sin evidencia adicional, la probabilidad de incendio es de 6.516 %. Cuando se incorpora una alarma activa, aumenta a 32.952 %, debido a que esta constituye una señal relacionada con la posible presencia de un incendio. Sin embargo, cuando también se conoce que existe una falla del sensor, la probabilidad disminuye a 7.683 %, ya que la falla puede explicar la activación de la alarma sin que necesariamente exista un incendio real.

B).
Independencia condicional en cadena_humo



Para verificar la independencia condicional se consulta la probabilidad de Humo=1 manteniendo Incendio=1 y variando el valor de Sequedad.

Primera consulta:

Variable: Humo.
Evidencia personalizada: Incendio=1,Sequedad=0

Resultado	Probabilidad obtenida
Humo=0	0.200000
Humo=1	0.800000

$$P(H = 1 | I = 1, S = 0) = 0.800000$$

Segunda consulta:

Variable: Humo.
Evidencia personalizada: Incendio=1,Sequedad=1

Resultado	Probabilidad obtenida
Humo=0	0.200000
Humo=1	0.800000

$$P(H = 1 | I = 1, S = 1) = 0.800000$$

Humo ⊥ Sequedad | Incendio

Independencia marginal
Sequedad = 0

Resultado	Probabilidad obtenida
Humo=0	0.886000
Humo=1	0.114000

Sequedad = 1

Resultado	Probabilidad obtenida
Humo=0	0.690000
Humo=1	0.310000

Aunque Humo y Sequedad son condicionalmente independientes cuando se conoce Incendio, no son marginalmente independientes. Esto se observa porque la probabilidad de Humo cambia de 0.114 a 0.310 al variar Sequedad sin observar Incendio. Esto ocurre porque Sequedad influye sobre Incendio, que a su vez afecta Humo.

Punto 3

A).

Llamemos $S = sequedad, I = Incendio$ y $H = humo$. Los datos son:

$$P(S = 1) = 0.4 \quad P(S = 0) = 0.6$$
$$P(I = 1|S = 0) = 0.02 \quad P(I = 1|S = 1) = 0.30$$
$$P(H = 1|I = 0) = 0.10 \quad P(H = 1|I = 1) = 0.80$$

La red se factoriza como:

$$P(S, I, H) = P(S)P(I|S)P(H|I)$$

Primero eliminamos Sequedad multiplicando los factores que contienen S:

$$g(S, I) = P(S)P(\text{I|S})$$

S	I	$g(S,I)$
0	0	0.588
0	1	0.012
1	0	0.280
1	1	0.120

Se suma sobre S, marginalizado las probabilidades de I:

$$f(I) = \sum_{s} g(s,I)$$

I	$f(I)$
0	0.868
1	0.132

Posteriormente se debe eliminar incendio con el sigueinte proceso:

$$k(I,H) = f(I)P(H|I)$$

I	H	$k(I,H)$
0	0	0.7812
0	1	0.0868
1	0	0.0264
1	1	0.1056

Ahora se suma sobre I:

H	$P(H)$
0	0.8076

1	0.1924

El resultado es:

$$P(H = 1) = 0.1924 = 19.24\%$$

B).
La suma de asignaciones con Alarma = 1 es:

$$P(I = 0|A = 1) = 0.121$$
$$P(I = 1|A = 1) = 0.059$$

Entonces:

$$P(A = 1) = 0.18$$

Finalmente:

$$P(I = 1|A = 1) = \frac{0.059}{0.18} = 0.329 = 32.9\%$$

Metodo Verificado	$P(I = 1\|A = 1)$	Maximo de filas
Suma del tabla conjunta	0.329	No Aplica
Eliminación: Topologico	0.329	8
Eliminación: Inverso	0.329	32

Paso	**Variable Eliminada: Topologico**	**Filas**	**Variable Eliminada: Inverso**	**Filas**
1	Tormenta	8	Temperatura	32
2	Sequedad	8	Humo	16
3	Falla	8	Falla	8
4	Humo	8	Sequedad	8
5	Temperatura	4	Tormenta	4

Punto 4

A).

Elemento	Descripción
Estados (S)	Posiciones transitables del dron en la cuadrícula
Acciones (A)	Norte (N), Este (E), Sur (S) y Oeste (W).
Transiciones (T)	Probabilidades de desplazamiento considerando la acción seleccionada y el viento.
Ruido	0.20, según el escenario cuadricula.
Recompensa por movimiento	-0.04
Estado terminal favorable	G: recompensa +1
Estado terminal desfavorable	X: recompensa -1
Factor de descuento	γ = 0.90.

B).

Valor esperado de las acciones

$$Q(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V(s')]$$

El valor de una acción se calcula considerando todos los posibles estados siguientes, sus probabilidades de transición, las recompensas inmediatas y el valor futuro descontado. De esta manera, se obtiene el beneficio esperado de realizar cada movimiento.

escenario control

Estado (x,y)	V final	Q(N)	Q(E)	Q(S)	Q(W)	Política
0,0	1.000000	0.860000	1.000000	0.860000	0.860000	Punto 5
1,0	0.000000	Terminal	Terminal	Terminal	Terminal	Terminal

Se observa que la acción Este presenta el mayor valor esperado (1.000000), debido a que permite alcanzar directamente el estado terminal favorable y obtener su recompensa. Las demás acciones presentan un valor de 0.860000, ya que no permiten llegar directamente a la descarga.

C).

Iteración de valores

$$V_{k+1}(s) = \max_a Q_k(s,a)$$

$$\delta = \max_s |V_{k+1}(s) - V_k(s)|$$

Se implementó el algoritmo de iteración de valores utilizando barridos sincrónicos, comenzando con $V_0(s)=0$ para todos los estados. En cada iteración se calculan los nuevos valores utilizando exclusivamente los valores del barrido anterior. Los estados terminales mantienen un valor de cero y el algoritmo se detiene cuando el cambio máximo entre dos iteraciones consecutivas es inferior a la tolerancia establecida.

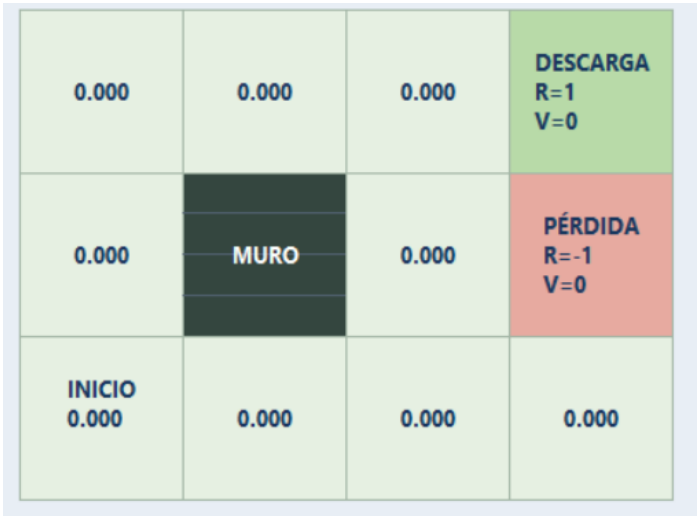
Barrido	Cambio máximo	V(inicio)	Cumple tolerancia
1	1	1.000000	No
2	0	1.000000	Sí

En el escenario control, el algoritmo alcanzó la convergencia después de dos barridos. El cambio máximo pasó de 1 a 0, cumpliendo el criterio de parada establecido. Esto permite comprobar el correcto funcionamiento de la actualización de valores.

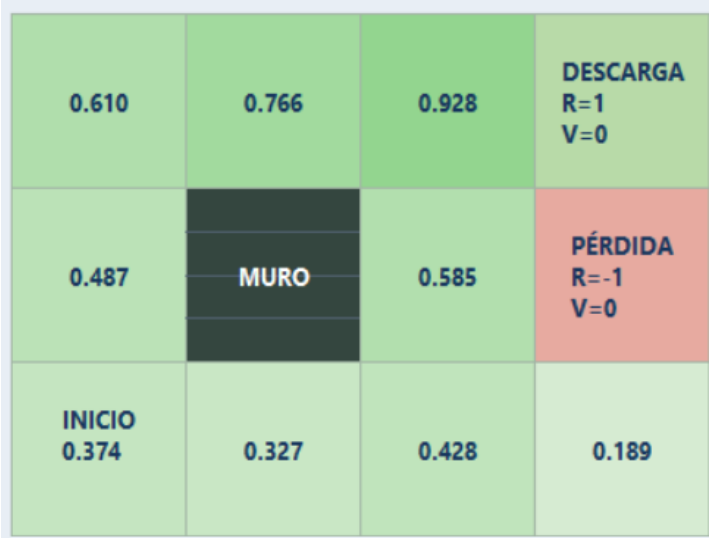
D).

Propagación de valores en cuadrícula

Valores iniciales:



Valores finales:



Se observa que inicialmente todos los estados comienzan con un valor de cero. A medida que se realizan los barridos, los valores se propagan desde los estados terminales hacia las demás posiciones. Las celdas cercanas a la descarga favorable presentan valores más altos, mientras que las posiciones con menor posibilidad de alcanzar una recompensa positiva presentan valores inferiores.

Tabla de valores finales:

Estado (x,y)	V final	Q(N)	Q(E)	Q(S)	Q(W)	Política
0,0	0.610462	0.523433	0.610462	0.434709	0.498325	Punto 5
1,0	0.766207	0.650147	0.766207	0.650147	0.537450	Punto 5
2,0	0.928180	0.801248	0.928180	0.554111	0.647849	Punto 5
3,0	0.000000	Terminal	Terminal	Terminal	Terminal	Terminal
0,1	0.487235	0.487235	0.399397	0.316875	0.399397	Punto 5
2,1	0.584934	0.584934	-0.685985	0.224475	0.503167	Punto 5
3,1	0.000000	Terminal	Terminal	Terminal	Terminal	Terminal
0,2	0.373852	0.373852	0.272666	0.292216	0.306671	Punto 5
1,2	0.326623	0.267294	0.326623	0.267294	0.287965	Punto 5
2,2	0.427543	0.427543	0.187077	0.314221	0.286291	Punto 5
3,2	0.188825	-0.752527	0.016948	0.151427	0.188825	Punto 5

Evolución de los primeros barridos

Barrido	Cambio máximo	V(inicio)	Cumple tolerancia
1	0.792	-0.040000	No
2	0.56304	-0.076000	No
3	0.3989088	-0.108400	No
4	0.281382336	-0.137560	No
5	0.220984278	0.083424	No
6	0.137125698	0.220550	No
7	0.0804858444	0.301036	No
8	0.0394240877	0.340460	No
9	0.0186481212	0.359108	No
10	0.00835313698	0.367461	No
11	0.00367049241	0.371132	No
12	0.00157434872	0.372706	No
13	0.000667910454	0.373374	No
14	0.000279766404	0.373654	No
15	0.000116421821	0.373770	No
16	4.80986634e-05	0.373818	No
17	1.97938089e-05	0.373838	No
18	8.11220023e-06	0.373846	No
19	3.31732874e-06	0.373849	No
20	1.3536271e-06	0.373851	No
21	5.51794686e-07	0.373851	No

Convergencia y criterio de parada

$$\delta = \max_{s} |V_{k+1}(s) - V_{k}(s)|$$

$$\delta < tolerance$$

Indicador	Resultado
Valor inicial final	26
Tolerancia	0.373852
Número de barridos	$6.91093816 \times 10^{-9}$
Delta final	1×10^{-8}
Convergencia	Sí

El algoritmo alcanzó la convergencia después de 26 barridos. El cambio máximo final fue de $6.91093816 \times 10^{-9}$, un valor inferior a la tolerancia establecida de 10^{-8}. Por lo tanto, se cumple el criterio de parada y el valor del estado inicial se estabiliza en 0.373852.

25	1.57694923e-08	0.373852	No
26	6.91093816e-09	0.373852	Sí

Punto 5

A). Para extraer la política óptima se implementó la función extract_policy en algorithms/planning.py. La función recorre todos los estados del MDP y, para cada estado no terminal, compara el valor esperado de las acciones disponibles utilizando action_value. En los estados terminales se asigna None.

B). Comparación de la política en el escenario atajo al modificar el ruido.
Primera prueba: ruido = 0.

```
Alerta Bosque | Punto 5 | Atajo con viento
Ruido=0.0 | Recompensa por movimiento=-0.04 | Descuento=0.95
Barridos=9 | Convergió: Sí | V(inicio)=1.366587
Acción de la política en el inicio: E

Valores finales y política:
Inicio: estado inicial | R: riesgo | T: terminal (V=0) | #: muro
N/E/S/W: dirección de la política
y / x | 0              | 1             | 2             | 3             | 4             | 5             | 6
------+----------------+---------------+---------------+---------------+---------------+---------------+-------------
0     | 1.155 E        | 1.258 E       | 1.367 E       | 1.481 E       | 1.601 E       | 1.727 E       | 1.860 S
1     | 1.258 E        | 1.367 E       | 1.481 E       | 1.601 E       | 1.727 E       | 1.860 E       | 2.000 S
2     | Inicio 1.367 E | 1.481 E       | 1.601 E       | 1.727 E       | 1.860 E       | 2.000 E       | T(R=2) 0.000
3     | 1.258 N        | T(R=-5) 0.000 | T(R=-5) 0.000 | T(R=-5) 0.000 | T(R=-5) 0.000 | T(R=-5) 0.000 | 2.000 N
```

Parámetro	Resultado
Ruido	0.0
Recompensa por movimiento	-0.04
Descuento	0.95
Barridos	9
Convergió	Sí
V(inicio)	1.366587
Acción inicial	E

Con ruido igual a 0, el dron se mueve exactamente en la dirección seleccionada. La política elige Este (E) desde el estado inicial y utiliza el camino más directo hacia el objetivo. Como no existe posibilidad de desviación, pasar cerca de las celdas de pérdida no representa un riesgo adicional.
Segunda prueba: ruido = 0.3.

```
Alerta Bosque | Punto 5 | Atajo con viento
Ruido=0.3 | Recompensa por movimiento=-0.04 | Descuento=0.95
Barridos=93 | Convergió: Sí | V(inicio)=0.711171
Acción de la política en el inicio: N

Valores finales y política:
Inicio: estado inicial | R: riesgo | T: terminal (V=0) | #: muro
N/E/S/W: dirección de la política
y / x | 0             | 1             | 2             | 3             | 4             | 5             | 6
------+---------------+---------------+---------------+---------------+---------------+---------------+-------------
0     | 0.826 E       | 0.954 E       | 1.089 E       | 1.232 E       | 1.380 E       | 1.529 E       | 1.677 S
1     | 0.801 E       | 0.936 E       | 1.087 E       | 1.256 E       | 1.446 E       | 1.659 E       | 1.894 S
2     | Inicio 0.711 N | 0.820 N       | 0.959 N       | 1.118 N       | 1.302 N       | 1.555 N       | T(R=2) 0.000
3     | 0.319 W       | T(R=-5) 0.000 | T(R=-5) 0.000 | T(R=-5) 0.000 | T(R=-5) 0.000 | T(R=-5) 0.000 | 1.382 E
```

Parámetro	Resultado
Ruido	0.3
Recompensa por movimiento	-0.04
Descuento	0.95
Barridos	93
Convergió	Sí
V(inicio)	0.711171
Acción inicial	N

Al aumentar el ruido a 0.3, la acción inicial cambia de Este (E) a Norte (N). Esto ocurre porque ahora existe una probabilidad de que el viento desvíe al dron. La política prefiere alejarse de las celdas X y utilizar un recorrido más seguro, aunque sea más largo.

El cambio de la acción inicial de E a N muestra que la política se vuelve más conservadora cuando aumenta la incertidumbre.

C). Comparación de la política en el escenario zona_riesgo modificando la recompensa por movimiento.
Primera prueba: living_reward = -0.04.

```
Alerta Bosque | Punto 5 | Sector de riesgo
Ruido=0.1 | Recompensa por movimiento=-0.04 | Descuento=0.95
Barridos=28 | Convergió: Sí | V(inicio)=1.003639
Acción de la política en el inicio: N

Valores finales y política:
Inicio: estado inicial | R: riesgo | T: terminal (V=0) | #: muro
N/E/S/W: dirección de la política
y / x | 0              | 1       | 2         | 3         | 4         | 5       | 6
------+----------------+---------+-----------+-----------+-----------+---------+-------------
0     | 1.104 E        | 1.221 E | 1.347 E   | 1.504 E   | 1.670 E   | 1.831 E | 1.977 S
1     | Inicio 1.004 N | 1.089 N | R 1.207 N | R 1.349 N | R 1.803 E | 1.970 E | T(R=2) 0.000
2     | 1.104 E        | 1.221 E | 1.347 E   | 1.504 E   | 1.670 E   | 1.831 E | 1.977 N
```

Parámetro	Resultado
Ruido	0.1
Recompensa por movimiento	-0.04
Descuento	0.95

Barridos	28
Convergió	Sí
V(inicio)	1.003639
Acción inicial	N

Con una penalización pequeña por movimiento, la política elige Norte (N). El dron puede realizar un recorrido más largo para evitar las celdas marcadas como riesgo, porque el costo de dar pasos adicionales es relativamente bajo.

Segunda prueba: living_reward = -0.30.

```
Alerta Bosque | Punto 5 | Sector de riesgo
Ruido=0.1 | Recompensa por movimiento=-0.3 | Descuento=0.95
Barridos=27 | Convergió: Sí | V(inicio)=-0.715505
Acción de la política en el inicio: E

Valores finales y política:
Inicio: estado inicial | R: riesgo | T: terminal (V=0) | #: muro
N/E/S/W: dirección de la política
y / x | 0              | 1         | 2           | 3           | 4           | 5       | 6
------+----------------+-----------+-------------+-------------+-------------+---------+-------------
0     | -0.435 E       | -0.094 E  | 0.271 E     | 0.681 E     | 1.097 E     | 1.516 E | 1.934 S
1     | Inicio -0.716 E | -0.438 N | R -0.079 N  | R 0.636 E   | R 1.441 E   | 1.914 E | T(R=2) 0.000
2     | -0.435 E       | -0.094 E  | 0.271 E     | 0.681 E     | 1.097 E     | 1.516 E | 1.934 N
```

Parámetro	Resultado
Ruido	0.1
Recompensa por movimiento	-0.30
Descuento	0.95
Barridos	27
Convergió	Sí
V(inicio)	-0.715505
Acción inicial	E

Al aumentar la penalización por movimiento, la acción inicial cambia de Norte (N) a Este (E). Como cada paso tiene un costo mayor, la política empieza a preferir trayectorias más directas, incluso cuando estas se acercan a la zona de riesgo.

Comparación del factor de descuento.

Primera prueba: descuento = 0.50.

```
Alerta Bosque | Punto 5 | Sector de riesgo
Ruido=0.1 | Recompensa por movimiento=-0.04 | Descuento=0.5
Barridos=19 | Convergió: Sí | V(inicio)=-0.073433
Acción de la política en el inicio: N

Valores finales y política:
Inicio: estado inicial | R: riesgo | T: terminal (V=0) | #: muro
N/E/S/W: dirección de la política
y / x | 0              | 1        | 2          | 3          | 4         | 5       | 6
------+----------------+----------+------------+------------+-----------+---------+-------------
0     | -0.066 E       | -0.050 E | -0.015 E   | 0.104 E    | 0.359 E   | 0.867 E | 1.864 S
1     | Inicio -0.073 N | -0.076 W | R -0.069 N | R -0.015 N | R 0.806 E | 1.839 E | T(R=2) 0.000
2     | -0.066 E       | -0.050 E | -0.015 E   | 0.104 E    | 0.359 E   | 0.867 E | 1.864 N
```

Parámetro	Resultado
Ruido	0.1
Recompensa por movimiento	-0.04
Descuento	0.50
Barridos	19
Convergió	Sí
V(inicio)	-0.073433
Acción inicial	N

Con un descuento de 0.50, las recompensas futuras tienen un menor peso en la decisión del agente. Por esta razón, aunque la política mantiene como acción inicial ir al Norte (N), el valor del estado inicial resulta negativo, ya que el beneficio de alcanzar el objetivo más adelante se ve reducido por el bajo factor de descuento.

Segunda prueba: descuento = 0.95.

```
Alerta Bosque | Punto 5 | Sector de riesgo
Ruido=0.1 | Recompensa por movimiento=-0.04 | Descuento=0.95
Barridos=28 | Convergió: Sí | V(inicio)=1.003639
Acción de la política en el inicio: N

Valores finales y política:
Inicio: estado inicial | R: riesgo | T: terminal (V=0) | #: muro
N/E/S/W: dirección de la política
y / x | 0              | 1        | 2          | 3          | 4         | 5       | 6
------+----------------+----------+------------+------------+-----------+---------+-------------
0     | 1.104 E        | 1.221 E | 1.347 E    | 1.504 E    | 1.670 E   | 1.831 E | 1.977 S
1     | Inicio 1.004 N | 1.089 N | R 1.207 N | R 1.349 N | R 1.803 E | 1.970 E | T(R=2) 0.000
2     | 1.104 E        | 1.221 E | 1.347 E    | 1.504 E    | 1.670 E   | 1.831 E | 1.977 N
```

Parámetro	Resultado
Ruido	0.1

Recompensa por movimiento	-0.04
Descuento	0.95
Barridos	28
Convergió	Sí
V(inicio)	1.003639
Acción inicial	N

Con descuento 0.95, las recompensas futuras conservan mayor importancia. El valor del estado inicial aumenta a 1.003639. Aunque la acción inicial permanece en Norte (N), la política valora mucho más la posibilidad de llegar al objetivo en el futuro.

Punto 6

A). Probabilidad y utilidad esperada

La evidencia modifica la probabilidad estimada de incendio y también puede modificar la decisión que se toma. En sensor_basico, sin evidencia la probabilidad de incendio es 10 % y la mejor decisión es inspeccionar. Cuando se recibe una alarma, la probabilidad aumenta a 66.67 % y la mejor decisión pasa a ser enviar la brigada. Cuando no hay alarma, la probabilidad de incendio baja aproximadamente a 1.16 % y la decisión es continuar monitoreando.

Evidencia	Probabilidad de incendio	Decisión
Sin evidencia	10 %	Inspeccionar
Alarma activa	66.67 %	Enviar brigada
Sin alarma	1.16 %	Monitorear

La probabilidad indica qué tan posible es que ocurra un incendio. La utilidad esperada también tiene en cuenta las consecuencias y los costos de cada acción. Por esto, la decisión final no depende solamente de la probabilidad, sino también de qué tan costoso sería equivocarse.

B). Enumeración y eliminación de variables

La enumeración y la eliminación de variables producen el mismo resultado porque ambas son técnicas de inferencia exacta y utilizan la misma red, las mismas probabilidades y la misma evidencia.

En el escenario bosque con Alarma = 1, ambos métodos obtuvieron una probabilidad de incendio de 32.9517 %. Sin embargo, el orden de eliminación sí afectó el tamaño de los factores intermedios.

Orden de eliminación	Mayor factor intermedio
Forward	8 filas
Reverse	32 filas

C). Muestreo por rechazo

Al aumentar el número de intentos, normalmente la aproximación se acerca al valor exacto. Para incendio sin evidencia, la probabilidad exacta era 6.516 %. Con 1.000 intentos se obtuvo 5.60 %, mientras que con 10.000 intentos se obtuvo 6.34 %.

Intentos	Estimación	Error aproximado
1.000	5.60 %	0.916 %
10.000	6.34 %	0.176 %

La probabilidad de la evidencia también influye en la cantidad de muestras aceptadas. Con Alarma = 1 se aceptaron 191 muestras de 1.000 intentos y 1.841 de 10.000. En cambio, para una evidencia rara se aceptaron 0 muestras de 1.000 intentos y solamente 11 de 10.000.

Evidencia	Intentos	Muestras aceptadas
Alarma = 1	1.000	191
Alarma = 1	10.000	1.841
Evidencia rara	1.000	0
Evidencia rara	10.000	11

D). Iteración de valores

Al principio, varios estados tienen valores negativos porque cada movimiento tiene un costo. A medida que se hacen más barridos, la recompensa de llegar al objetivo empieza a afectar también a los estados que están más lejos.

Barrido	V(inicio) aproximado
1	-0.040000
2	-0.076000
3	-0.108400
4	-0.137560
5	0.083424
6	0.220550
7	0.301036
26	0.373852

En el escenario cuadricula, el algoritmo necesitó 26 barridos para estabilizarse. Se detiene cuando el cambio entre un barrido y el siguiente ya es muy pequeño, según la tolerancia definida. En otras palabras, cuando los valores casi no cambian, se considera que el algoritmo ya llegó a una solución estable.

E). Efecto de los parámetros y diferencia entre política y ruta fija

Los resultados del Punto 5 muestran que los parámetros del MDP cambian el comportamiento del dron.

Cambio	Resultado principal
Ruido 0 -> 0.3	La acción inicial cambia de E a N.
living_reward -0.04 -> -0.30	La acción inicial cambia de N a E.
Descuento 0.50 -> 0.95	La acción inicial sigue en N, pero V(inicio) cambia de -0.073433 a 1.003639.

Cuando aumenta el ruido, el dron tiene más posibilidades de desviarse, por eso la política empieza a evitar caminos cercanos a zonas peligrosas. Si el costo de cada movimiento aumenta, el dron busca hacer menos pasos y trata de ir por rutas más directas. En cuanto al descuento, cuando es alto, llegar al objetivo en el futuro sigue siendo importante; cuando es bajo, pesan más las consecuencias de los movimientos que se hacen en ese momento.

Una política no es una ruta fija que el dron tenga que seguir desde el principio. Más bien, le indica qué hacer dependiendo de la posición en la que se encuentre. Si el viento lo mueve a otro lugar, el dron vuelve a consultar la política y decide qué movimiento hacer desde ahí. Por eso, usando la misma política, el recorrido puede terminar siendo diferente.

6b. Reflexión sobre el uso de inteligencia artificial

Angela:
Durante el desarrollo del taller utilicé la inteligencia artificial como apoyo para comprender mejor conceptos, revisar la implementación de los algoritmos y resolver las dudas que tenía sobre los resultados de las distintas pruebas. El uso de inteligencia artificial debe ser algo que se maneje con cuidado debido a que no debe hacerse un mal uso o uso excesivo usandolo como única respuesta o usarla sin revisar la información que nos está dando.
Utilicé la IA para aclarar conceptos, interpretar resultados y mejorar la redacción del documento, lo que permitió una presentación más clara. Verifiqué soluciones ejecutando algoritmos en Alerta Bosque, comparando resultados, y comprendí procedimientos para explicar las soluciones en sustentaciones.

Uso de inteligencia artificial
Punto 2:
Codigo inicial:

```
FUNCIÓN enumeration_ask(red, consulta, evidencia):

   Validar la consulta y la evidencia.

   FUNCIÓN enumerate_all(indice, evento):

      SI se recorrieron todas las variables:
         RETORNAR 1

      Obtener la variable actual.

      SI la variable está en el evento:
         Calcular su probabilidad.
         Multiplicar por el resultado recursivo.

      SI NO:
         Inicializar total = 0.

         PARA cada valor posible (0 y 1):
```

Crear una copia del evento.
Asignar el valor a la variable.
Calcular su probabilidad.
Sumar el resultado recursivo.

RETORNAR total.

PARA cada valor posible de la consulta:
Agregar el valor a una copia de la evidencia.
Calcular la enumeración correspondiente.

Comprobar que la evidencia sea posible.

Normalizar los resultados.

RETORNAR la distribución de probabilidades.

Promt:

¿Cómo debería quedar el algoritmo de enumeración manteniendo como base mi codigo inicial y que cumpla con la actividad y el retorno indicado?

Codigo final:

```
# TODO: Add your code here

   def enumerate_all(index, event):
       if index == len(network.variables):
           return 1.0

       variable = network.variables[index]

       if variable in event:
           probability = network.probability(
               variable, event[variable], event
           )
           return probability * enumerate_all(index + 1, event)

       total = 0.0

       for value in (0, 1):
           extended = event.copy()
           extended[variable] = value

           probability = network.probability(
               variable, value, extended
           )
```

```
            total += probability * enumerate_all(
                index + 1, extended
            )

        return total


    weights = {}

    for value in (0, 1):
        if query in evidence and evidence[query] != value:
            weights[value] = 0.0
            continue

        event = dict(evidence)
        event[query] = value

        weights[value] = enumerate_all(0, event)

    total = sum(weights.values())

    if not math.isfinite(total) or total <= 0.0:
        raise ValueError("La evidencia es imposible")

    return normalize(weights)

# FIN DEL BLOQUE DEL ESTUDIANTE
```

Aceptamos los cambios sugeridos, la implementación se verificó ejecutando el escenario sensor_basico y comparando el resultado con el cálculo manual mediante la regla de Bayes. Posteriormente, se realizaron pruebas en bosque y cadena_humo, comprobando las distribuciones obtenidas y las relaciones de independencia condicional y marginal.

Punto 4:

Codigo inicial:

FUNCIÓN action_value(mdp, estado, acción, valores):

total = 0

PARA cada estado_siguiente y su probabilidad:
Obtener la recompensa.
Calcular recompensa inmediata
más valor futuro descontado.
Multiplicar por la probabilidad.

Acumular en total.

RETORNAR total.

FUNCIÓN value_iteration(mdp):

Inicializar V(s) = 0 para todos los estados.

REPETIR hasta convergencia o máximo de iteraciones:

Crear un diccionario de nuevos valores.

PARA cada estado:
SI es terminal:
Asignar valor 0.
SI NO:
Calcular el valor de todas las acciones.
Seleccionar el mayor valor.

Calcular el cambio máximo entre
los valores anteriores y los nuevos.

Actualizar los valores.

Comprobar si el cambio máximo
es menor que la tolerancia.

RETORNAR los resultados y el historial.

Pomt:
Explícame cómo implementar action_value y value_iteration paso a paso teniendo en cuenta este código inicial como base para la solución.
Codigo Final:

```
# TODO: Add your code here
        new_values = {}

        for state in mdp.states:
            available_actions = mdp.actions(state)

            if not available_actions:
                new_values[state] = 0.0

            else:
                new_values[state] = max(
                    action_value(mdp, state, action, values)
                    for action in available_actions
                )

        delta = max(
            abs(new_values[state] - values[state])
```

```
            for state in mdp.states
        )

        values = new_values
        converged = delta < tolerance
        # FIN DEL BLOQUE DEL ESTUDIANTE
        # Copias para la animación y empaquetado del resultado.
        history.append(values.copy())
        deltas.append(delta)
        if converged:
            return PlanningResult(values, history, deltas, True)
    return PlanningResult(values, history, deltas, False)
```

Aceptamos la solución propuesta, luego se verificó mediante la interfaz gráfica de Alerta Bosque, utilizando los escenarios control y cuadrícula. En control se comprobaron los valores de las acciones y la convergencia después de dos barridos. En cuadrícula se revisó la propagación de valores, el historial de iteraciones y el cumplimiento del criterio de convergencia después de 26 barridos.
También se compararon los valores finales de los estados y el valor del estado inicial con los resultados mostrados por la aplicación.

Mariana:
Durante el desarrollo del taller utilicé inteligencia artificial como herramienta de apoyo para revisar la lógica de mi implementación, entender algunos errores y comprobar que el comportamiento del algoritmo correspondiera con lo solicitado en el enunciado. La IA fue útil principalmente para identificar problemas en la selección de la mejor acción y en el manejo de los empates.

Al revisar las sugerencias entendí que no era correcto inicializar el mejor valor en cero, porque algunas acciones pueden tener valores negativos. También aprendí que usar >= podía alterar el orden de desempate solicitado, ya que podía reemplazar una acción anterior aunque tuviera el mismo valor.

No acepté todas las sugerencias inicialmente. Por ejemplo, se propuso utilizar math.isclose con tolerancias para comparar números decimales, pero preferí buscar una solución más sencilla

de comprender y explicar. Posteriormente, al realizar las pruebas, se observó que podían aparecer diferencias muy pequeñas por precisión numérica, por lo que fue necesario tener en cuenta este comportamiento al verificar los empates. La solución se verificó ejecutando los escenarios atajo y zona_riesgo con diferentes valores de ruido, recompensa por movimiento y descuento.

Punto 5

Código inicial:

```
def extract_policy(mdp, values):
    policy = {}

    for state in mdp.states:
        actions = mdp.actions(state)

        if not actions:
            policy[state] = None
            continue

        best_action = actions[0]
        best_value = 0

        for action in actions:
            value = action_value(mdp, state, action, values)

            if value >= best_value:
                best_value = value
                best_action = action

        policy[state] = best_action

    return policy
```

Prompt utilizado:

Hola chat, tengo este código para mi función extract_policy, pero se que tiene algunos errores. Por lo que, te quería preguntar como puedo corregir el código manteniendo mi idea respetando el desempate N, E, S, W que pide el taller. Aquí te dejo la documentación que nos da unos tips.

```
"""Punto 5: retorna acción por estado; terminales -> None.

    Entradas: mdp es el modelo y values = {(x, y): valor} viene del punto 4.
    Salida: {(x, y): acción}, usando N/E/S/W o None para terminales.
```

Tips:
- Reutilice action_value: no basta elegir el vecino de mayor valor.
- Respete mdp.actions: desempate N, E, S, W conservando el primero.
"""

Cambios sugeridos por la IA:

Se identificaron principalmente tres problemas:

1. `best_value` no debía inicializarse en `0`, porque una acción podía tener un valor negativo.
2. No era necesario volver a evaluar `actions[0]`, ya que esta acción ya se utilizaba como referencia inicial.
3. El uso de `>=` podía cambiar la acción en caso de empate, por lo que debía utilizarse `>` para conservar la primera acción según el orden N, E, S, W.

Código corregido:

```python
def extract_policy(mdp, values):
    policy = {}

    for state in mdp.states:
        actions = mdp.actions(state)

        if not actions:
            policy[state] = None
            continue

        best_action = actions[0]
        best_value = action_value(
            mdp,
            state,
            best_action,
            values
        )

        for action in actions[1:]:
            value = action_value(
                mdp,
                state,
                action,
                values
            )
```

```
            if value > best_value:
                best_value = value
                best_action = action

        policy[state] = best_action

    return policy
```

Acepté los cambios relacionados con la inicialización de best_value, el recorrido desde actions[1:] y el cambio de >= por >, porque estos corregían errores de lógica y permitían respetar el orden de desempate solicitado. Y la implementación se verificó ejecutando varias pruebas del Punto 5.